

GRUPO 26 – Correcciones Trabajo Práctico: ÁRBOLES

RESULTADO DE LA CORRECCIÓN: DESAPROBADO

OBSERVACIONES

Se detectan resoluciones idénticas a las realizadas por el Grupo 21

```
ArbolBinario a_modos_carga(){
    int opcion, cantNodo;
    ArbolBinario arbol;

    printf("Seleccione el modo de carga del arbol\n");
    printf(" 1. Carga manual por anchura\n");
    printf(" 2. Carga al azar con valores unicos\n");
    printf(" 3. Carga al azar con valores que pueden repetirse\n");
    printf("> Seleccione una opcion: ");
    opcion = ingresoIntLimitado("Ingrese 1, 2 o 3");

    if (opcion == 1){
        printf("\n\n");
        arbol = a_cargar_anchura();
        limpiarPantalla();
    }
    else if (opcion == 2){
        printf("\n\n");
        printf("Ingrese la cantidad de nodos que desea cargar: ");
        cantNodo = ingresoDatosNumericosPositivos("Cantidad de nodos");
        printf("\n\n");
        arbol = a_cargar_azar(cantNodo);
    }
    else{
        printf("\n\n");
        printf("Ingrese la cantidad de nodos que desea cargar: ");
        cantNodo = ingresoDatosNumericosPositivos("Cantidad de nodos");
        printf("\n\n");
        arbol = a_cargar_azar_rep(cantNodo);
    }

    return arbol;
}

ArbolBinario nArio_cargar_anchura(){
    ArbolBinario arbol = a_crear();
    int clave, clave_hi, clave_hd;
    char buffer[32];
}
```

```
ArbolBinario a_cargar_azar(int cantidad_nodos) {
    if (cantidad_nodos <= 0) return a_crear();

    ArbolBinario arbol = a_crear();
    NodoArbol* nodos = malloc(sizeof(NodoArbol) * cantidad_nodos);
    int claves[cantidad_nodos];
    int usados = 0;
    srand((unsigned int)time(NULL));

    // Generar claves únicas aleatorias
    for (int i = 0; i < cantidad_nodos; ++i) {
        int clave;
        int unico;
        do {
            clave = 1 + rand() % (100 - 1 + 1);
            unico = 1;
            for (int j = 0; j < i; ++j) {
                if (claves[j] == clave) {
                    unico = 0;
                    break;
                }
            }
        } while (!unico);
        claves[i] = clave;
    }

    // Insertar la raíz
    nodos[0] = a_establecer_raiz(arbol, te_crear(cantidad_nodos, claves));
    usados = 1;

    int insertados = 1;
    while (insertados < cantidad_nodos) {
        // Elegir nodo padre al azar entre los ya insertados
        int padre_idx = rand() % usados;
        NodoArbol padre = nodos[padre_idx];

        // Insertar como hijo izquierdo si no tiene izquierdo
        if (n_hijoizquierdo(padre) == NULL) {
            nodos[usados++] = a_conectar_hijo_izquierdo(padre, te_crear(1, claves[usados]));
            insertados++;
        }
        // Insertar como hijo derecho si no tiene derecho
        else if (n_hijodercho(padre) == NULL) {
            nodos[usados++] = a_conectar_hijo_dercho(padre, te_crear(1, claves[usados]));
            insertados++;
        }
    }

    return arbol;
}
```

GRUPO 26 – Correcciones Trabajo Práctico: ÁRBOLES

```
ArbolBinario nArio_cargar_azar(int cantidad_nodos) {
    if (cantidad_nodos <= 0) {
        return a_crear();
    }

    ArbolBinario arbol = a_crear();
    NodoArbol* nodos = malloc(sizeof(NodoArbol) * cantidad_nodos);
    int claves[cantidad_nodos];
    int usados = 0;
    srand((unsigned int)time(NULL));

    // Generar claves únicas aleatorias
    for (int i = 0; i < cantidad_nodos; ++i) {
        int clave, unico;
        do {
            clave = 1 + rand() % (100 - 1 + 1);
            unico = 1;
            for (int j = 0; j < i; ++j) {
                if (claves[j] == clave) {
                    unico = 0;
                    break;
                }
            }
        } while (!unico);
        claves[i] = clave;
    }

    // Insertar la raíz
    nodos[0] = a_establecer_raiz(arbol, te_crear(cantidad_nodos, claves, usados));
    usados = 1;

    int insertados = 1;
    while (insertados < cantidad_nodos) {
        // Elegir nodo padre al azar entre los ya insertados
        int padre_idx = rand() % usados;
        NodoArbol padre = nodos[padre_idx];

        // Insertar como hijo izquierdo si no tiene hijo izquierdo
        if (n_hijoizquierdo(padre) == NULL) {
            nodos[usados++] = a_conectar_hijo_izquierdo(arbol, padre, te_crear(cantidad_nodos - insertados, claves, usados));
            insertados++;
        }
    }

    return arbol;
}
```

```
ArbolBinario nArio_modos_carga() {
    int opcion, cant_nodo;
    ArbolBinario arbol;

    printf("Seleccione el modo de carga del árbol:\n");
    printf(" 1. Carga manual por anchura\n");
    printf(" 2. Carga al azar con valores únicos\n");
    printf(" 3. Carga al azar con valores que pueden repetirse\n");
    printf("> Seleccione una opción: ");
    opcion = ingresoIntLimitado("Ingrese 1, 2 o 3");

    if (opcion == 1) {
        printf("\n\n");
        arbol = nArio_cargar_anchura();
        system("cls");
    } else if (opcion == 2) {
        printf("\n\n");
        printf("Ingrese la cantidad de nodos que desea cargar: ");
        cant_nodo = ingresoDatosNumericosPositivos("Cantidad de nodos");
        printf("\n\n");
        arbol = nArio_cargar_azar(cant_nodo);
    } else {
        printf("\n\n");
        printf("Ingrese la cantidad de nodos que desea cargar: ");
        cant_nodo = ingresoDatosNumericosPositivos("Cantidad de nodos");
        printf("\n\n");
        arbol = nArio_cargar_azar_rep(cant_nodo);
    }

    return arbol;
}
```


GRUPO 26 – Correcciones Trabajo Práctico: ÁRBOLES

The image shows a code editor with two files open: `validaciones.c` and `nArio_modos.c`. The left pane shows the Explorer view with a file tree for `GRUPO21-26`. The main editor area displays the code for `validaciones.c`, which includes functions for loading a binary tree (`ArbolBinario nArio_modos_carga`) and a recursive function for displaying the tree (`ArbolBinario mostrar_normal`). The code is written in C and uses `printf` for output and `system` for clearing the screen.

```
1190 ArbolB: 26/Grupo26/grupo26alrichappe/programacion2-
1191 re: 2026/libros/validaciones/implementaciones/validaciones.c
1192
1193
1194 ArbolBinario nArio_modos_carga(){
1195     int opcion, cant_nodo;
1196     ArbolBinario arbol;
1197
1198     printf("Seleccione el modo de carga del arbol\n");
1199     printf(" 1. Carga manual por anchura\n");
1200     printf(" 2. Carga al azar con valores unicos\n");
1201     printf(" 3. Carga al azar con valores que pueden repetirse\n");
1202     printf("> Seleccione una opcion: ");
1203     opcion = ingresoIntLimitado("Ingrese 1, 2 o 3");
1204     if (opcion == 1) {
1205         printf("\n\n");
1206         arbol = nArio_cargar_anchura();
1207         system("cls");
1208     } else if (opcion == 2) {
1209         printf("\n\n");
1210         printf("Ingrese la cantidad de nodos que desea cargar: ");
1211         cant_nodo = ingresoDatosNumericosPositivos("Ingrese cantidad de nodos");
1212         printf("\n\n");
1213         arbol = nArio_cargar_azar(cant_nodo);
1214     } else {
1215         printf("\n\n");
1216         printf("Ingrese la cantidad de nodos que desea cargar: ");
1217         cant_nodo = ingresoDatosNumericosPositivos("Ingrese cantidad de nodos");
1218         printf("\n\n");
1219         arbol = nArio_cargar_azar_rep(cant_nodo);
1220     }
1221     return arbol;
1222 }
1223
1224 ArbolBinario nArio_modos_carga(){
1225     int opcion, cant_nodo;
1226     ArbolBinario arbolito;
1227
1228     printf("Seleccione el modo de carga del arbol\n");
1229     printf(" 1. Carga manual por anchura\n");
1230     printf(" 2. Carga al azar con valores unicos\n");
1231     printf(" 3. Carga al azar con valores que pueden repetirse\n");
1232     printf("> Seleccione una opcion: ");
1233     opcion = ingresoIntLimitado("Ingrese 1, 2 o 3");
1234     if (opcion == 1){
1235         printf("\n\n");
1236         arbolito = nArio_cargar_anchura();
1237         system("cls");
1238     } else if (opcion == 2){
1239         printf("\n\n");
1240         printf("Ingrese la cantidad de nodos que desea cargar: ");
1241         cant_nodo = ingresoDatosNumericosPositivos("Ingrese cantidad de nodos");
1242         printf("\n\n");
1243         arbolito = nArio_cargar_azar(cant_nodo);
1244     } else{
1245         printf("\n\n");
1246         printf("Ingrese la cantidad de nodos que desea cargar: ");
1247         cant_nodo = ingresoDatosNumericosPositivos("Ingrese cantidad de nodos");
1248         printf("\n\n");
1249         arbolito = nArio_cargar_azar_rep(cant_nodo);
1250     }
1251     return arbolito;
1252 }
1253
1254 ArbolBinario nArio_cargar_azar_rep(int cantidad_nodos){
1255     if (cantidad_nodos <= 0) return a_crear(1);
1256     ArbolBinario arbolito;
1257     arbolito = nArio_cargar_azar_rep(cantidad_nodos - 1);
1258     arbolito->dato = ingresoDatosNumericosPositivos("Ingrese dato para el nodo");
1259     arbolito->hijo_izquierdo = nArio_cargar_azar_rep(cantidad_nodos - 1);
1260     arbolito->hijo_derecho = nArio_cargar_azar_rep(cantidad_nodos - 1);
1261     return arbolito;
1262 }
```

grapo26arilchappe / programacion2-2026 / libs / validaciones / implementaciones

validaciones.c

Code Blame

Raw Copy Download Edit Close

```
652
653
654 ArbolBinario a_modocarga(){
655     int opcion, cant_nodo;
656     ArbolBinario arbolito;
657
658     printf("Seleccione el modo de carga del arbol binario:\n");
659     printf(" 1. Carga manual por anchura\n");
660     printf(" 2. Carga al azar con valores unicos\n");
661     printf(" 3. Carga al azar con valores que pueden repetirse\n");
662     printf("> Seleccione una opcion: ");
663     opcion = ingresointimitado("Ingrese 1, 2 o 3", 1, 3);
664     if (opcion == 1){
665         printf("\n\n");
666         arbolito = a_cargar_anchura();
667         system("cls");
668     }
669     else if (opcion == 2){
670         printf("\n\n");
671         printf("Ingrese la cantidad de nodos que desea cargar en el arbol o <=0 para cargar cant_nodo = ingresodatosNumericosPositivos("Ingrese un valor adecuado!");
672         printf("\n\n");
673         arbolito = a_cargar_azar(cant_nodo);
674     }
675     else{
676         printf("\n\n");
677         printf("Ingrese la cantidad de nodos que desea cargar en el arbol o <=0 para cargar cant_nodo = ingresodatosNumericosPositivos("Ingrese un valor adecuado!");
678         printf("\n\n");
679         arbolito = a_cargar_azar_rep(cant_nodo);
680     }
681     return arbolito;
682 }
```

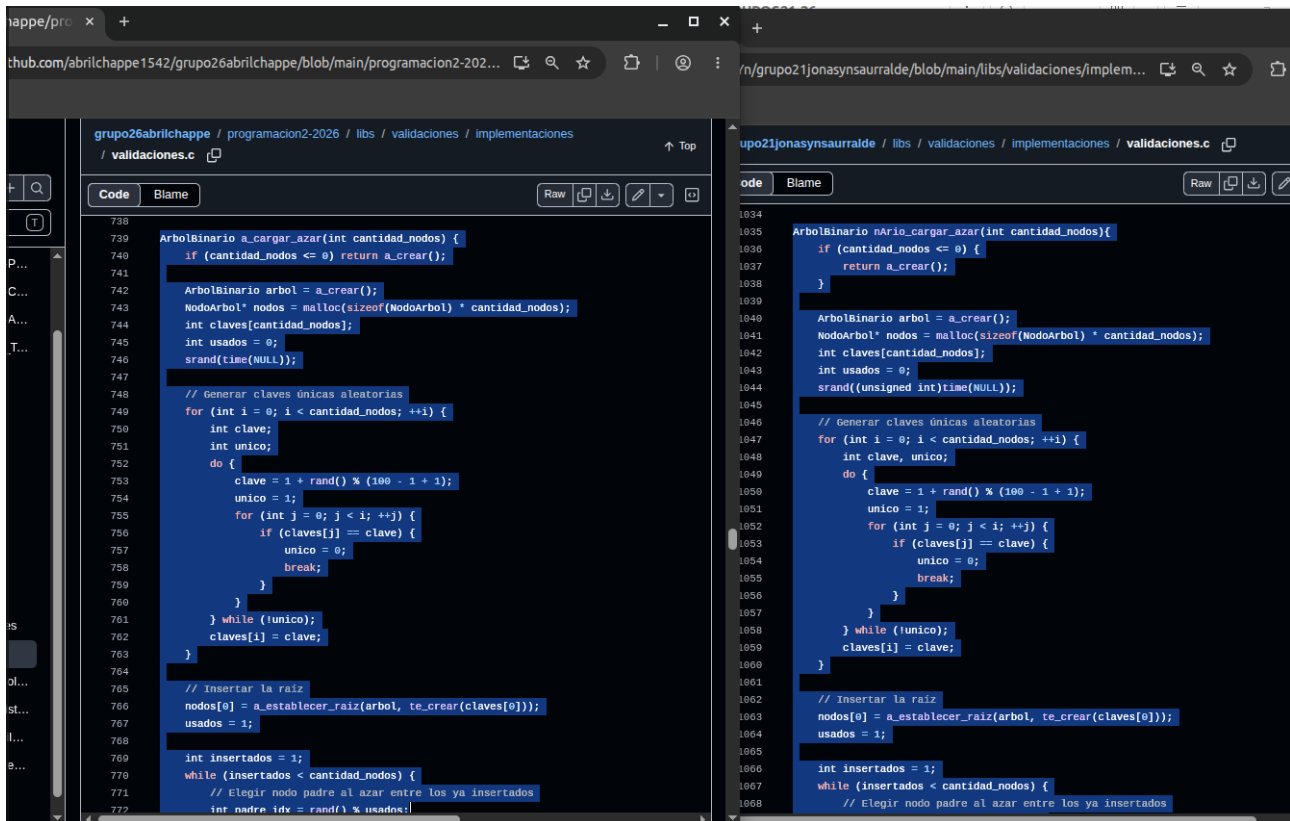
grapo21jonasynsauralde / libs / validaciones / implementaciones / validaciones.c

Code Blame

Raw Copy Download Edit Close

```
952
953 ArbolBinario a_modocarga(){
954     int opcion, cantNodo;
955     ArbolBinario arbol;
956
957     printf("Seleccione el modo de carga del arbol binario:\n");
958     printf(" 1. Carga manual por anchura\n");
959     printf(" 2. Carga al azar con valores unicos\n");
960     printf(" 3. Carga al azar con valores que pueden repetirse\n");
961     printf("> Seleccione una opcion: ");
962     opcion = ingresointimitado("Ingrese 1, 2 o 3", 1, 3);
963     if (opcion == 1){
964         printf("\n\n");
965         arbol = a_cargar_anchura();
966         limpiarPantalla();
967     }
968     else if (opcion == 2){
969         printf("\n\n");
970         printf("Ingrese la cantidad de nodos que desea cargar en el arbol o <=0 para cargar cantNodo = ingresodatosNumericosPositivos("Ingrese un valor adecuado!");
971         printf("\n\n");
972         arbol = a_cargar_azar(cantNodo);
973     }
974     else{
975         printf("\n\n");
976         printf("Ingrese la cantidad de nodos que desea cargar en el arbol o <=0 para cargar cantNodo = ingresodatosNumericosPositivos("Ingrese un valor adecuado!");
977         printf("\n\n");
978         arbol = a_cargar_azar_rep(cantNodo);
979     }
980     return arbol;
981 }
```

GRUPO 26 – Correcciones Trabajo Práctico: ÁRBOLES



The image shows two side-by-side code editors. The left editor, titled 'grupo26abrilchappe', displays the function 'ArbolBinario a_cargar_azar(int cantidad_nodos)'. It includes logic for generating unique keys, creating nodes, and inserting them into the tree. The right editor, titled 'grupo21jonasynsauralde', shows a similar function 'ArbolBinario nArio_cargar_azar(int cantidad_nodos)'. Both functions handle key generation, node creation, and tree insertion.

```
ArbolBinario a_cargar_azar(int cantidad_nodos) {
    if (cantidad_nodos <= 0) return a_crear();

    ArbolBinario arbol = a_crear();
    NodoArbol* nodos = malloc(sizeof(NodoArbol) * cantidad_nodos);
    int claves[cantidad_nodos];
    int usados = 0;
    srand(time(NULL));

    // Generar claves unicas aleatorias
    for (int i = 0; i < cantidad_nodos; ++i) {
        int clave;
        int unico;
        do {
            clave = 1 + rand() % (100 - 1 + 1);
            unico = 1;
            for (int j = 0; j < i; ++j) {
                if (claves[j] == clave) {
                    unico = 0;
                    break;
                }
            } while (!unico);
            claves[i] = clave;
        }
    }

    // Insertar la raiz
    nodos[0] = a_establecer_raiz(arbol, te_crear(claves[0]));
    usados = 1;

    int insertados = 1;
    while (insertados < cantidad_nodos) {
        // Elegir nodo padre al azar entre los ya insertados
        int padre_idx = rand() % usados;
        int padre_idx = rand() % usados;
    }
}
```

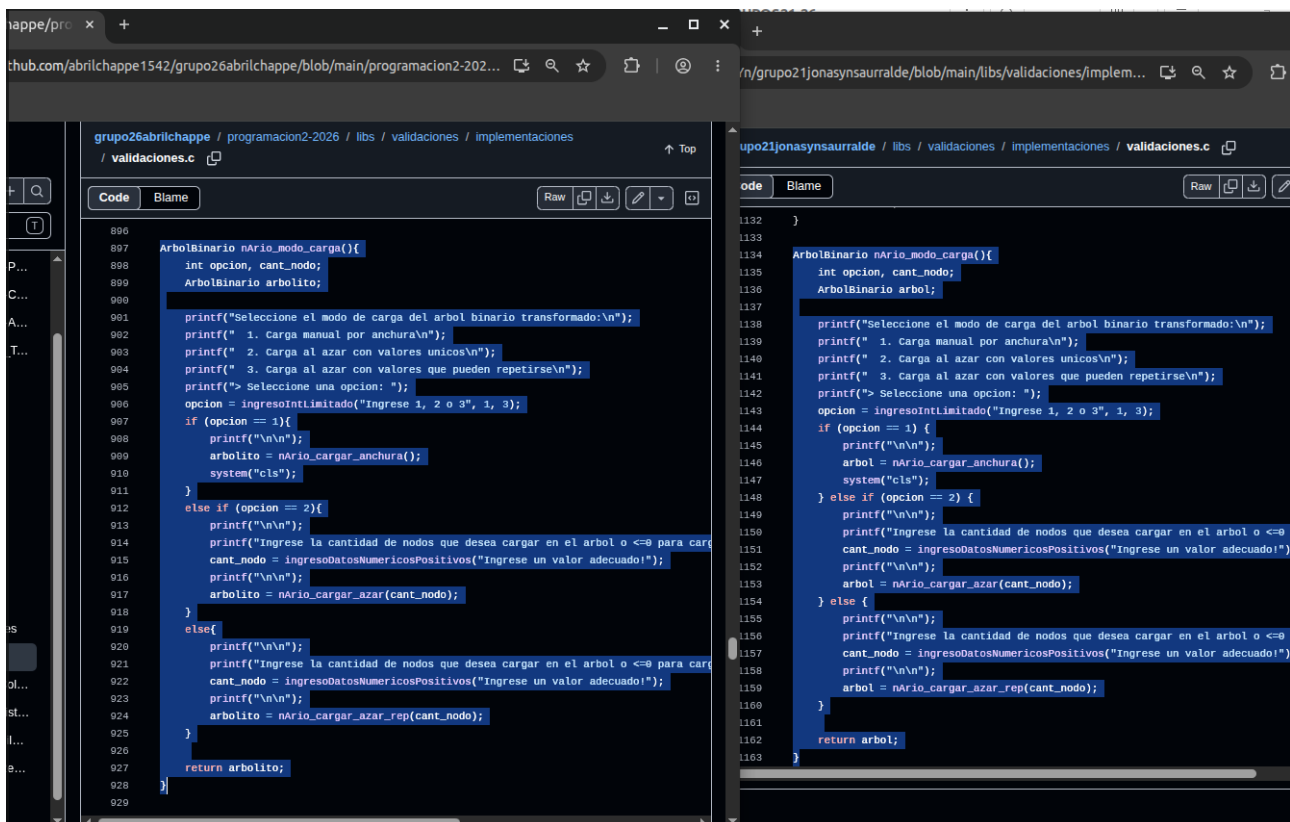
```
ArbolBinario nArio_cargar_azar(int cantidad_nodos){
    if (cantidad_nodos <= 0) {
        return a_crear();
    }

    ArbolBinario arbol = a_crear();
    NodoArbol* nodos = malloc(sizeof(NodoArbol) * cantidad_nodos);
    int claves[cantidad_nodos];
    int usados = 0;
    srand((unsigned int)time(NULL));

    // Generar claves unicas aleatorias
    for (int i = 0; i < cantidad_nodos; ++i) {
        int clave, unico;
        do {
            clave = 1 + rand() % (100 - 1 + 1);
            unico = 1;
            for (int j = 0; j < i; ++j) {
                if (claves[j] == clave) {
                    unico = 0;
                    break;
                }
            } while (!unico);
            claves[i] = clave;
        }
    }

    // Insertar la raiz
    nodos[0] = a_establecer_raiz(arbol, te_crear(claves[0]));
    usados = 1;

    int insertados = 1;
    while (insertados < cantidad_nodos) {
        // Elegir nodo padre al azar entre los ya insertados
    }
}
```



The image shows two side-by-side code editors. The left editor, titled 'grupo26abrilchappe', displays the function 'ArbolBinario nArio_modos_carga()'. It includes a menu for selecting the loading mode (manual, unique values, or random values) and handles the corresponding logic. The right editor, titled 'grupo21jonasynsauralde', shows a similar function 'ArbolBinario nArio_modos_carga()'. Both functions handle menu selection, input validation, and tree loading.

```
ArbolBinario nArio_modos_carga(){
    int opcion, cant_nodo;
    ArbolBinario arbolito;

    printf("Seleccione el modo de carga del arbol binario transformado:\n");
    printf(" 1. Carga manual por anchura\n");
    printf(" 2. Carga al azar con valores unicos\n");
    printf(" 3. Carga al azar con valores que pueden repetirse\n");
    printf("Seleccione una opcion: ");
    opcion = ingresoIntLimitado("Ingrese 1, 2 o 3", 1, 3);

    if (opcion == 1){
        printf("\n\n");
        arbolito = nArio_cargar_anchura();
        system("cls");
    }
    else if (opcion == 2){
        printf("\n\n");
        printf("Ingrese la cantidad de nodos que desea cargar en el arbol o <=0 para cargar todos los nodos: ");
        cant_nodo = ingresoDatosNumericosPositivos("Ingrese un valor adecuado!");
        printf("\n\n");
        arbolito = nArio_cargar_azar(cant_nodo);
    }
    else{
        printf("\n\n");
        printf("Ingrese la cantidad de nodos que desea cargar en el arbol o <=0 para cargar todos los nodos: ");
        cant_nodo = ingresoDatosNumericosPositivos("Ingrese un valor adecuado!");
        printf("\n\n");
        arbolito = nArio_cargar_azar_rep(cant_nodo);
    }

    return arbolito;
}
```

```
ArbolBinario nArio_modos_carga(){
    int opcion, cant_nodo;
    ArbolBinario arbol;

    printf("Seleccione el modo de carga del arbol binario transformado:\n");
    printf(" 1. Carga manual por anchura\n");
    printf(" 2. Carga al azar con valores unicos\n");
    printf(" 3. Carga al azar con valores que pueden repetirse\n");
    printf("> Seleccione una opcion: ");
    opcion = ingresoIntLimitado("Ingrese 1, 2 o 3", 1, 3);

    if (opcion == 1) {
        printf("\n\n");
        arbol = nArio_cargar_anchura();
        system("cls");
    }
    else if (opcion == 2) {
        printf("\n\n");
        printf("Ingrese la cantidad de nodos que desea cargar en el arbol o <=0 para cargar todos los nodos: ");
        cant_nodo = ingresoDatosNumericosPositivos("Ingrese un valor adecuado!");
        printf("\n\n");
        arbol = nArio_cargar_azar(cant_nodo);
    }
    else {
        printf("\n\n");
        printf("Ingrese la cantidad de nodos que desea cargar en el arbol o <=0 para cargar todos los nodos: ");
        cant_nodo = ingresoDatosNumericosPositivos("Ingrese un valor adecuado!");
        printf("\n\n");
        arbol = nArio_cargar_azar_rep(cant_nodo);
    }

    return arbol;
}
```

GRUPO 26 – Correcciones Trabajo Práctico: ÁRBOLES

```

C: menu.c x
> grupo26abrilchappe > programacion2-2026 > 05-trabajoPractico-Arboles > C: menu.c > menuPunto7()
276 // EJERCICIO 7
277 void menuPunto7(){
278     int opcion, cant_nodo;
279     bool seguir = true, resultado;
280     while (seguir == 1){
281         printf("\n=== PUNTO 7 ===\n");
282
283         printf("\nCargamos primer arbol\n\n");
284         ArbolBinario arbolito1 = a_modos_carga();
285         system("cls");
286         printf("\n|| GRAFICO DEL ARBOL 1 ||\n\n");
287         a_mostrar_normal(arbolito1);
288         if(a_es_vacio(arbolito1)) printf("\n\n");
289
290         printf("\nCargamos segundo arbol\n\n");
291         ArbolBinario arbolito2 = a_modos_carga();
292         system("cls");
293         printf("\n|| GRAFICO DEL ARBOL 2 ||\n\n");
294         a_mostrar_normal(arbolito2);
295         if(a_es_vacio(arbolito2)) printf("\n\n");
296
297         system("cls");
298         printf("\n|| GRAFICO DEL ARBOL 1 ||\n\n");
299         a_mostrar_normal(arbolito1);
300         if(a_es_vacio(arbolito1)) printf("\n\n");
301         printf("\n\n\n|| GRAFICO DEL ARBOL 2 ||\n\n");
302         a_mostrar_normal(arbolito2);
303         if(a_es_vacio(arbolito2)) printf("\n\n");
304
305         resultado = a_ej7_equivalente(arbolito1, arbolito2);
306         if(resultado == true){
307             if(a_es_vacio(arbolito1) && a_es_vacio(arbolito2))
308                 printf("\n\nLos dos arboles estaban vacios\n\n");
309             printf("\n=== LOS ARBOLES SON EQUIVALENTES ===\n\n");
310         }
311         else{
312             printf("\n=== LOS ARBOLES NO SON EQUIVALENTES ===\n\n");
313         }
314         seguir = preguntarContinuar();
315     }
}

C: menu.c x
> grupo21jonasynsauralde > 05-trabajoPractico-Arboles > C: menu.c > menuPunto07()
263 void menuPunto04() {
374
375     void menuPunto07() {
376         int opcion;
377         bool seguir = true, resultado;
378         while (seguir) {
379             printf("\n=== PUNTO 7 ===\n");
380
381             printf("\nCargamos primer arbol\n\n");
382             ArbolBinario arbol1 = a_modos_carga();
383             limpiarPantalla();
384             printf("\n|| GRAFICO DEL ARBOL 1 ||\n\n");
385             a_mostrar_normal(arbol1);
386             if (a_es_vacio(arbol1)) {
387                 printf("\n\n");
388             }
389             printf("\nCargamos segundo arbol\n\n");
390             ArbolBinario arbol2 = a_modos_carga();
391             limpiarPantalla();
392             printf("\n|| GRAFICO DEL ARBOL 2 ||\n\n");
393             a_mostrar_normal(arbol2);
394             if (a_es_vacio(arbol2)) {
395                 printf("\n\n");
396             }
397             limpiarPantalla();
398             printf("\n|| GRAFICO DEL ARBOL 1 ||\n\n");
399             a_mostrar_normal(arbol1);
400             if (a_es_vacio(arbol1)) {
401                 printf("\n\n");
402             }
403
404             printf("\n\n\n|| GRAFICO DEL ARBOL 2 ||\n\n");
405             a_mostrar_normal(arbol2);
406             if (a_es_vacio(arbol2)) {
407                 printf("\n\n");
408             }
409
410             resultado = a_ej7_equivalente(arbol1, arbol2);
411             if (resultado == true) {

```

```
C menu.c x
> grupo26abrilchappe > programacion2-2026 > 05-trabajoPractico-Arboles > C menu.c > menuPunto07()
277 void menuPunto7(){
280     while (seguir == 1){
291         ArbolBinario arbolito2 = a_modos_carga();
292         system("cls");
293         printf("\n|| GRAFICO DEL ARBOL 2 ||\n\n");
294         a_mostrar_normal(arbolito2);
295         if(a_es_vacio(arbolito2)) printf("\n\n");
296
297         system("cls");
298         printf("\n|| GRAFICO DEL ARBOL 1 ||\n\n");
299         a_mostrar_normal(arbolito1);
300         if(a_es_vacio(arbolito1)) printf("\n\n");
301         printf("\n\n\n|| GRAFICO DEL ARBOL 2 ||\n\n");
302         a_mostrar_normal(arbolito2);
303         if(a_es_vacio(arbolito2)) printf("\n\n");
304
305         resultado = a_ej7_equivalente(arbolito1, arbolito2);
306         if(resultado == true){
307             if(a_es_vacio(arbolito1) && a_es_vacio(arbolito2)
308                 printf("\n\nLos dos arboles estaban vacios\n");
309             }
310             printf("\n=== LOS ARBOLES SON EQUIVALENTES ===\n");
311         }
312         else{
313             printf("\n=== LOS ARBOLES NO SON EQUIVALENTES ===\n");
314         }
315         seguir = preguntarContinuar();
316         system("cls");
317     }
318 }
319
320 // EJERCICIO 8
321 void menuPunto8(){
322     int opcion, cant_nodo, altura, nivel, clave;
323     Lista l_res;
324     bool seguir = true;
325
326     ArbolBinario arbolito = nArio_modos_carga();
327     system("cls");
328     printf("\n|| GRAFICO DEL ARBOL ||\n\n");
329
330     Grupo21 > grupo21jonasynsauralde > 05-trabajoPractico-Arboles > C menu.c > menuPunto07()
331 void menuPunto07() {
332     while (seguir) {
333         if (a_es_vacio(arbol2)) {
334             limpiarPantalla();
335             printf("\n|| GRAFICO DEL ARBOL 1 ||\n\n");
336             a_mostrar_normal(arbol1);
337             if (a_es_vacio(arbol1)) {
338                 printf("\n\n");
339             }
340             printf("\n\n\n|| GRAFICO DEL ARBOL 2 ||\n\n");
341             a_mostrar_normal(arbol2);
342             if (a_es_vacio(arbol2)) {
343                 printf("\n\n");
344             }
345             resultado = a_ej7_equivalente(arbol1, arbol2);
346             if (resultado == true) {
347                 if (a_es_vacio(arbol1) && a_es_vacio(arbol2)) {
348                     printf("\n\nLos dos arboles estaban vacios\n");
349                 }
350                 printf("\n=== LOS ARBOLES SON EQUIVALENTES ===\n");
351             } else {
352                 printf("\n=== LOS ARBOLES NO SON EQUIVALENTES ===\n");
353             }
354             seguir = preguntarContinuar();
355             limpiarPantalla();
356         }
357     }
358
359     void menuPunto08() {
360         int opcion, altura, nivel, clave;
361         Lista lista;
362         bool seguir = true;
363
364         ArbolBinario arbol = nArio_modos_carga();
365         limpiarPantalla();
366         printf("\n|| GRAFICO DEL ARBOL ||\n\n");
367         a_mostrar_normal(arbol);
368     }
369 }
```


GRUPO 26 – Correcciones Trabajo Práctico: ÁRBOLES

```
86 // EJERCICIO 3
87 void menuPunto03(){
88     int opcion, cant_nodo;
89     bool seguir = true;
90     int clave1, clave2;
91     Lista l;
92     Iterador iter;
93     TipoElemento te;
94
95     ArbolBinario arbolito = a_modos_carga();
96     system("cls");
97     printf("\n|| GRAFICO DEL ARBOL ||\n\n");
98     a_mostrar_normal(arbolito);
99     if(a_es_vacio(arbolito)) printf("\n\n");
100
101     while (seguir == 1){
102         printf("\n=== MENU DEL PUNTO 3 ===\n");
103         printf("1. Ejercicio A (Indicar el nombre del nodo padre (clave))\n");
104         printf("2. Ejercicio B (Listar los hijos (solo las clave/s))\n");
105         printf("3. Ejercicio C (Listar el hermano (solo la clave))\n");
106         printf("4. Ejercicio D (Calcular el nivel en el que se encuentra)\n");
107         printf("5. Ejercicio E (Calcular la altura de su rama (Altura del Subárbol))\n");
108         printf("6. Ejercicio F (Listar todos los nodos que estan en el mismo nivel (solo los que estan en el mismo nivel))\n");
109         printf("0. Salir\n");
110         printf("-1. Cargar nuevo arbol\n");
111         printf("> Seleccione una opcion: ");
112         opcion = ingresoIntLimitado("Ingrese un valor adecuado!", -1, 0);
113         system("cls");
114
115         l = l_crear();
116         switch (opcion){
117             case -1:
118                 arbolito = a_modos_carga();
119                 system("cls");
120                 printf("\n|| GRAFICO DEL ARBOL ||\n\n");
121                 a_mostrar_normal(arbolito);
122                 if(a_es_vacio(arbolito)) printf("\n\n");
123                 break;
124             case 0:
125                 seguir = 0;
126                 break;
127             default:
128                 break;
129         }
130     }
131 }
132
133 void menuPunto07(){
134     ArbolBinario arbol = a_modos_carga();
135     limpiarPantalla();
136
137     printf("\n|| GRAFICO DEL ARBOL ||\n\n");
138     a_mostrar_normal(arbol);
139     if (a_es_vacio(arbol)) {
140         printf("\n\n");
141     }
142
143     while (seguir) {
144         printf("\n=== MENU DEL PUNTO 3 ===\n");
145         printf("2. Ejercicio B (Listar los hijos (solo las clave/s))\n");
146         printf("3. Ejercicio C (Listar el hermano (solo la clave))\n");
147         printf("4. Ejercicio D (Calcular el nivel en el que se encuentra)\n");
148         printf("5. Ejercicio E (Calcular la altura de su rama (Altura del Subárbol))\n");
149         printf("6. Ejercicio F (Listar todos los nodos que estan en el mismo nivel (solo los que estan en el mismo nivel))\n");
150         printf("0. Salir\n");
151         printf("-1. Cargar nuevo arbol\n");
152         printf("> Seleccione una opcion: ");
153         opcion = ingresoIntLimitado("Ingrese un valor adecuado!", -1, 0);
154         limpiarPantalla();
155
156         lista = l_crear();
157         switch (opcion) {
158             case -1:
159                 arbol = a_modos_carga();
160                 limpiarPantalla();
161                 printf("\n|| GRAFICO DEL ARBOL ||\n\n");
162                 a_mostrar_normal(arbol);
163                 if (a_es_vacio(arbol)) {
164                     printf("\n\n");
165                 }
166                 break;
167             default:
168                 break;
169         }
170     }
171 }
```

```
85 // EJERCICIO 3
86 void menuPuntos03(){
87     int opcion, cant_nodo;
88     bool seguir = true;
89     int clave1, clave2;
90     Lista l;
91     Iterador iter;
92     TipoElemento te;
93
94     ArbolBinario arbolito = a_modos_carga();
95     system("cls");
96     printf("\n|| GRAFICO DEL ARBOL ||\n\n");
97     a_mostrar_normal(arbolito);
98     if(a_es_vacio(arbolito)) printf("\n\n");
99
100     while (seguir == 1){
101         printf("\n=== MENU DEL PUNTO 3 ===\n");
102         printf("1. Ejercicio A (Indicar el nombre del nodo padre (clave))\n");
103         printf("2. Ejercicio B (Listar los hijos (solo las clave/s))\n");
104         printf("3. Ejercicio C (Listar el hermano (solo la clave))\n");
105         printf("4. Ejercicio D (Calcular el nivel en el que se encuentra)\n");
106         printf("5. Ejercicio E (Calcular la altura de su rama (Altura del Subárbol))\n");
107         printf("6. Ejercicio F (Listar todos los nodos que estan en el mismo nivel (solo los que estan en el mismo nivel))\n");
108         printf("0. Salir\n");
109         printf("-1. Cargar nuevo arbol\n");
110         printf("> Seleccione una opcion: ");
111         opcion = ingresoIntLimitado("Ingrese un valor adecuado!", -1, 0);
112         system("cls");
113
114         l = l_crear();
115         switch (opcion){
116             case -1:
117                 arbolito = a_modos_carga();
118                 system("cls");
119                 printf("\n|| GRAFICO DEL ARBOL ||\n\n");
120                 a_mostrar_normal(arbolito);
121                 if(a_es_vacio(arbolito)) printf("\n\n");
122                 break;
123             case 0:
124                 seguir = 0;
125                 break;
126             default:
127                 break;
128         }
129     }
130 }
131
132 void menuPunto03(){
133     bool seguir = true;
134     int opcion, clave1, clave2;
135     Lista lista;
136
137     ArbolBinario arbol = a_modos_carga();
138     limpiarPantalla();
139
140     printf("\n|| GRAFICO DEL ARBOL ||\n\n");
141     a_mostrar_normal(arbol);
142     if (a_es_vacio(arbol)) {
143         printf("\n\n");
144     }
145
146     while (seguir) {
147         printf("\n=== MENU DEL PUNTO 3 ===\n");
148         printf("2. Ejercicio B (Listar los hijos (solo las clave/s))\n");
149         printf("3. Ejercicio C (Listar el hermano (solo la clave))\n");
150         printf("4. Ejercicio D (Calcular el nivel en el que se encuentra)\n");
151         printf("5. Ejercicio E (Calcular la altura de su rama (Altura del Subárbol))\n");
152         printf("6. Ejercicio F (Listar todos los nodos que estan en el mismo nivel (solo los que estan en el mismo nivel))\n");
153         printf("0. Salir\n");
154         printf("-1. Cargar nuevo arbol\n");
155         printf("> Seleccione una opcion: ");
156         opcion = ingresoIntLimitado("Ingrese un valor adecuado!", -1, 0);
157         limpiarPantalla();
158
159         lista = l_crear();
160         switch (opcion) {
161             case -1:
162                 arbol = a_modos_carga();
163                 limpiarPantalla();
164                 printf("\n|| GRAFICO DEL ARBOL ||\n\n");
165                 a_mostrar_normal(arbol);
166                 if (a_es_vacio(arbol)) {
167                     printf("\n\n");
168                 }
169                 break;
170             default:
171                 break;
172         }
173     }
174 }
```

GRUPO 26 – Correcciones Trabajo Práctico: ÁRBOLES

```
grupo26abrilchappe / programacion2-2026 / 05-trabajoPractico-Arboles / menu.c
Code Blame
276 // EJERCICIO 7
277 void menuPunto7(){
278     int opcion, cant_nodo;
279     bool seguir = true, resultado;
280     while (seguir == 1){
281         printf("\n=== PUNTO 7 ===\n");
282
283         printf("\nCargamos primer arbol\n\n");
284         ArbolBinario arbolito1 = a_modos_carga();
285         system("cls");
286         printf("\n| GRAFICO DEL ARBOL 1 |\n\n");
287         a_mostrar_normal(arbolito1);
288         if(a_es_vacio(arbolito1)) printf("\n\n");
289
290         printf("\nCargamos segundo arbol\n\n");
291         ArbolBinario arbolito2 = a_modos_carga();
292         system("cls");
293         printf("\n| GRAFICO DEL ARBOL 2 |\n\n");
294         a_mostrar_normal(arbolito2);
295         if(a_es_vacio(arbolito2)) printf("\n\n");
296
297         system("cls");
298         printf("\n| GRAFICO DEL ARBOL 1 |\n\n");
299         a_mostrar_normal(arbolito1);
300         if(a_es_vacio(arbolito1)) printf("\n\n");
301         printf("\n\n| GRAFICO DEL ARBOL 2 |\n\n");
302         a_mostrar_normal(arbolito2);
303         if(a_es_vacio(arbolito2)) printf("\n\n");
304
305         resultado = a_ej7_equivalente(arbolito1, arbolito2);
306         if(resultado == true){
307             if(a_es_vacio(arbolito1) && a_es_vacio(arbolito2)){
308                 printf("\n\nLos dos arboles estaban vacios\n");
309             }
310             printf("\n=== LOS ARBOLES SON EQUIVALENTES ===\n");
311         }
    }
}

jonasynsauralde / 05-trabajoPractico-Arboles / menu.c
Blame
void menuPunto7() {
    int opcion;
    bool seguir = true, resultado;
    while (seguir) {
        printf("\n=== PUNTO 7 ===\n");

        printf("\nCargamos primer arbol\n\n");
        ArbolBinario arbol1 = a_modos_carga();
        limpiarPantalla();
        printf("\n| GRAFICO DEL ARBOL 1 |\n\n");
        a_mostrar_normal(arbol1);
        if (a_es_vacio(arbol1)) {
            printf("\n\n");
        }
        printf("\nCargamos segundo arbol\n\n");
        ArbolBinario arbol2 = a_modos_carga();
        limpiarPantalla();
        printf("\n| GRAFICO DEL ARBOL 2 |\n\n");
        a_mostrar_normal(arbol2);
        if (a_es_vacio(arbol2)) {
            printf("\n\n");
        }
        limpiarPantalla();
        printf("\n| GRAFICO DEL ARBOL 1 |\n\n");
        a_mostrar_normal(arbol1);
        if (a_es_vacio(arbol1)) {
            printf("\n\n");
        }

        printf("\n\n| GRAFICO DEL ARBOL 2 |\n\n");
        a_mostrar_normal(arbol2);
        if (a_es_vacio(arbol2)) {
            printf("\n\n");
        }

        resultado = a_ej7_equivalente(arbol1, arbol2);
    }
}
```

```
grupo26abrilchappe / programacion2-2026 / 05-trabajoPractico-Arboles / menu.c
Code Blame
304
305     resultado = a_ej7_equivalente(arbolito1, arbolito2);
306     if(resultado == true){
307         if(a_es_vacio(arbolito1) && a_es_vacio(arbolito2)){
308             printf("\n\nLos dos arboles estaban vacios\n");
309         }
310         printf("\n=== LOS ARBOLES SON EQUIVALENTES ===\n");
311     }
312     else{
313         printf("\n=== LOS ARBOLES NO SON EQUIVALENTES ===\n");
314     }
315     seguir = preguntarContinuar();
316     system("cls");
317 }
318 }
319
320 // EJERCICIO 8
321 void menuPunto8(){
322     int opcion, cant_nodo, altura, nivel, clave;
323     Lista l_res;
324     bool seguir = true;
325
326     ArbolBinario arbolito = nArio_modos_carga();
327     system("cls");
328     printf("\n| GRAFICO DEL ARBOL |\n\n");
329     a_mostrar_normal(arbolito);
330     if(a_es_vacio(arbolito)) printf("\n\n");
331
332     while (seguir == 1){
333         printf("\n=== MENU DEL PUNTO 8 ===\n");
334         printf("1. Ejercicio A (Determinar la altura del n-ario)\n");
335         printf("2. Ejercicio B (Determinar el nivel de un nodo)\n");
336         printf("3. Ejercicio C (Listar todos los nodos internos (solo las claves))\n");
337         printf("4. Ejercicio D (Determinar si todas las hojas estan al mismo nivel)\n");
338         printf("0. Salir\n");
339         printf("-1. Cargar nuevo arbol\n");
340         printf("\nSeleccione una opcion: ");
    }
}

jonasynsauralde / 05-trabajoPractico-Arboles / menu.c
Blame
    resultado = a_ej7_equivalente(arbol1, arbol2);
    if (resultado == true) {
        if (a_es_vacio(arbol1) && a_es_vacio(arbol2)) {
            printf("\n\nLos dos arboles estaban vacios\n");
        }
        printf("\n=== LOS ARBOLES SON EQUIVALENTES ===\n");
    } else {
        printf("\n=== LOS ARBOLES NO SON EQUIVALENTES ===\n");
    }
    seguir = preguntarContinuar();
    limpiarPantalla();
}

void menuPunto8() {
    int opcion, altura, nivel, clave;
    Lista lista;
    bool seguir = true;

    ArbolBinario arbol = nArio_modos_carga();
    limpiarPantalla();
    printf("\n| GRAFICO DEL ARBOL |\n\n");
    a_mostrar_normal(arbol);
    if (a_es_vacio(arbol)) {
        printf("\n\n");
    }

    while (seguir) {
        printf("\n=== MENU DEL PUNTO 8 ===\n");
        printf("1. Ejercicio A (Determinar la altura del n-ario)\n");
        printf("2. Ejercicio B (Determinar el nivel de un nodo)\n");
        printf("3. Ejercicio C (Listar todos los nodos internos (solo las claves))\n");
        printf("4. Ejercicio D (Determinar si todas las hojas estan al mismo nivel)\n");
        printf("0. Salir\n");
        printf("-1. Cargar nuevo arbol\n");
        printf("> Seleccione una opcion: ");
        opcion = ingresoIntLimitado("Ingrese un valor adecuado!", -1, 4);
    }
}
```

GRUPO 26 – Correcciones Trabajo Práctico: ÁRBOLES

```
grupo26abrilchappe / programacion2-2026 / 05-trabajoPractico-Arboles / menu.c
331
332 while (seguir == 1){
333     printf("\n== MENU DEL PUNTO 8 ==\n");
334     printf("1. Ejercicio A (Determinar la altura del n-ario)\n");
335     printf("2. Ejercicio B (Determinar el nivel de un nodo)\n");
336     printf("3. Ejercicio C (Listar todos los nodos internos (solo las claves))\n");
337     printf("4. Ejercicio D (Determinar si todas las hojas estan al mismo nivel)\n");
338     printf("0. Salir \n");
339     printf("-1. Cargar nuevo arbol \n");
340     printf("> Seleccione una opcion: ");
341     opcion = ingresoIntLimitado("Ingrese un valor adecuado!", -1, 4);
342     system("cls");
343
344     switch (opcion){
345     case 1: // ejercicio a
346         printf("\n|| GRAFICO DEL ARBOL ||\n\n");
347         a_mostrar_normal(arbolito);
348         if(a_es_vacio(arbolito)) printf("\n\n");
349         altura = a_ej8_altura(arbolito);
350         if(altura == 0){
351             printf("\n\nEl arbol ingresado estaba vacio...");
352         }
353         else if(altura == 1){
354             printf("\n\nEl arbol ingresado solo tenia la raiz...");
355         }
356         printf("\n\nALTURA DEL ARBOL N-ARIO: %d\n", altura);
357         break;
358
359     case 2: // ejercicio b
360         printf("\n|| GRAFICO DEL ARBOL ||\n\n");
361         a_mostrar_normal(arbolito);
362         if(a_es_vacio(arbolito)) printf("\n\n");
363
364         printf("\n\nIngrese la clave del nodo que desea saber su nivel\n-> ");
365         clave = ingresoDatosNumericos("Ingrese un valor adecuado!");
366
```

```
jonasynsauralde / 05-trabajoPractico-Arboles / menu.c
}
while (seguir) {
    printf("\n== MENU DEL PUNTO 8 ==\n");
    printf("1. Ejercicio A (Determinar la altura del n-ario)\n");
    printf("2. Ejercicio B (Determinar el nivel de un nodo)\n");
    printf("3. Ejercicio C (Listar todos los nodos internos (solo las claves))\n");
    printf("4. Ejercicio D (Determinar si todas las hojas estan al mismo nivel)\n");
    printf("0. Salir \n");
    printf("-1. Cargar nuevo arbol \n");
    printf("> Seleccione una opcion: ");
    opcion = ingresoIntLimitado("Ingrese un valor adecuado!", -1, 4);
    limpiarPantalla();

    switch (opcion) {
    case 1: // ejercicio a
        printf("\n|| GRAFICO DEL ARBOL ||\n\n");
        a_mostrar_normal(arbol);
        if (a_es_vacio(arbol)) {
            printf("\n\n");
        }
        altura = a_ej8_altura(arbol);
        if (altura == 0) {
            printf("\n\nEl arbol ingresado estaba vacio...");
        } else if (altura == 1) {
            printf("\n\nEl arbol ingresado solo tenia la raiz...");
        }
        printf("\n\nALTURA DEL ARBOL N-ARIO: %d\n", altura);
        break;

    case 2: // ejercicio b
        printf("\n|| GRAFICO DEL ARBOL ||\n\n");
        a_mostrar_normal(arbol);
        if (a_es_vacio(arbol)) {
            printf("\n\n");
        }
        printf("\n\nIngrese la clave del nodo que desea saber su nivel\n-> ");

```

```
grupo26abrilchappe / programacion2-2026 / 05-trabajoPractico-Arboles / menu.c
357         break;
358
359     case 2: // ejercicio b
360         printf("\n|| GRAFICO DEL ARBOL ||\n\n");
361         a_mostrar_normal(arbolito);
362         if(a_es_vacio(arbolito)) printf("\n\n");
363
364         printf("\n\nIngrese la clave del nodo que desea saber su nivel\n-> ");
365         clave = ingresoDatosNumericos("Ingrese un valor adecuado!");
366
367         nivel = a_ej8_nivel(arbolito, clave);
368         if(nivel == -1){
369             printf("\n\nNo se encontro el nodo...");
370         }
371         else{
372             if(nivel == 0){
373                 printf("\n\nEl nodo era la raiz...");
374             }
375             printf("\n\nNIVEL DEL NODO %d EN EL ARBOL N-ARIO: %d\n", clave, nivel);
376         }
377         break;
378
379     case 3: // ejercicio c
380         printf("\n|| GRAFICO DEL ARBOL ||\n\n");
381         a_mostrar_normal(arbolito);
382         if(a_es_vacio(arbolito)) printf("\n\n");
383
384         l_res = a_ej8_internos(arbolito);
385         if(l_res_vacia(l_res)){
386             printf("\n\nNODOS INTERIORES DEL N-ARIO:\n");
387             printf("\n\nLa lista esta vacia...");
388         }
389         else{
390             printf("\n\nNODOS INTERIORES DEL N-ARIO:\n");
391             l_mostrar(l_res);
392         }
393
```

```
jonasynsauralde / 05-trabajoPractico-Arboles / menu.c
}
while (seguir) {
    printf("\n|| GRAFICO DEL ARBOL ||\n\n");
    a_mostrar_normal(arbol);
    if (a_es_vacio(arbol)) {
        printf("\n\n");
    }

    printf("\n\nIngrese la clave del nodo que desea saber su nivel\n-> ");
    clave = ingresoDatosNumericos("Ingrese un valor adecuado!");

    nivel = a_ej8_nivel(arbol, clave);
    if (nivel == -1) {
        printf("\n\nNo se encontro el nodo...");
    } else {
        if (nivel == 0) {
            printf("\n\nEl nodo era la raiz...");
        }
        printf("\n\nNIVEL DEL NODO %d EN EL ARBOL N-ARIO: %d\n", clave, nivel);
    }
    break;

    case 3: // ejercicio c
        printf("\n|| GRAFICO DEL ARBOL ||\n\n");
        a_mostrar_normal(arbol);
        if (a_es_vacio(arbol)) {
            printf("\n\n");
        }
        lista = a_ej8_internos(arbol);
        if (l_res_vacia(lista)) {
            printf("\n\nNODOS INTERIORES DEL N-ARIO:\n");
            printf("\n\nLa lista esta vacia...");
        } else {
            printf("\n\nNODOS INTERIORES DEL N-ARIO:\n");
            l_mostrar(lista);
        }
    }
}

```


GRUPO 26 – Correcciones Trabajo Práctico: ÁRBOLES

```
397 printf("\n|| GRAFICO DEL ARBOL ||\n\n");
398 a_mostrar_normal(arbolito);
399 if(a_es_vacio(arbolito)) printf("\n\n");
400
401 bool res = a_eje_hojasmismonivel(arbolito);
402 if(!a_es_vacio(arbolito)){
403     printf(res ? "\nLas hojas son del mismo nivel" : "\nLas hojas NO son
404 }
405 } else{
406     printf("\nEl arbol ingresado estaba vacio...");
407 }
408
409 break;
410
411 case 0:
412     printf("Seguro que desea salir?");
413     seguir = preguntarContinuar();
414     break;
415
416 case -1:
417     arbolito = nArio_moda_carga();
418     system("cls");
419     printf("\n|| GRAFICO DEL ARBOL ||\n\n");
420     a_mostrar_normal(arbolito);
421     if(a_es_vacio(arbolito)) printf("\n\n");
422     break;
423
424 default:
425     printf("AVISO: Ingrese un numero parte de las opciones.\n");
426     break;
427 }
428 printf("\n\n");
429 system("pause");
430 system("cls");
431 }
432 }
433
```

```

a_mostrar_normal(arbol);
if (a_es_vacio(arbol)) printf("\n\n");

bool res = a_eje_hojasmismonivel(arbol);
if (!a_es_vacio(arbol)) {
    printf(res ? "\nLas hojas son del mismo nivel" : "\nLas hojas NO son d
} else {
    printf("\nEl arbol ingresado estaba vacio...");
}

break;

case 0:
    printf("Seguro que desea salir?");
    seguir = !preguntarContinuar();
    break;

case -1:
    arbol = nArio_moda_carga();
    limpiarPantalla();
    printf("\n|| GRAFICO DEL ARBOL ||\n\n");
    a_mostrar_normal(arbol);
    if (a_es_vacio(arbol)) {
        printf("\n\n");
    }
    break;

default:
    printf("AVISO: Ingrese un numero parte de las opciones.\n");
    break;
}
seguir = preguntarContinuar();
limpiarPantalla();
}
}
```